

SC4053 BLOCKCHAIN TECHNOLOGY

Development Project

1 General Information

In the Development Project, you are expected to design and develop a decentralized Application (dApp) using tokens and smart contracts on Ethereum. This is a group activity (2–3 students per team) and accounts for 35% of the total course marks. Single-member groups will need explicit permission from the course coordinator.

2 AI Integration and AI Audit Report

To reflect modern software engineering practices, students are permitted and encouraged to utilize AI agents, specialized coding tools, and Large Language Models (LLMs) to assist in architecting, coding, testing, and auditing their dApps. However, submitting only code is no longer sufficient. You must critically review the AI-generated applications and address potential issues, either manually or with AI assistance.

Along with your code submission, your team is required to submit an accompanying **AI Audit Report**.

2.1 AI Audit Report Required Sections

Your report must include the following sections:

1. **Initial Specification & System Scope:** A description of the core requirements, smart contract logic, and UI features you intended to build before engaging AI tools.
2. **Development Methodology & AI Tooling Infrastructure:** A detailed breakdown of your development workflow and the toolchain used, including:
 - The AI coding assistants, IDE plugins, or agentic tools utilized (e.g., Claude Code, Kiro, Cursor, GitHub Copilot).
 - Any custom AI agents, automated scripts, specification files (`spec.md`, design blueprints), or LLM workflows constructed by your team to aid in contract generation, front-end binding, or automated testing.

- How AI was integrated into your team's division of labor and decision-making process.
3. **Specification Engineering & Prompt Logs:** How requirements were communicated to AI tools:
 - Formal specification files written for AI coding tools/agents.
 - Representative prompt chains, iterations, and context windows provided during generation.
 4. **AI-Generated Architecture:** An overview of the structural design, contract interfaces, and state management initially proposed by the AI tools/agents.
 5. **Problems Discovered & Vulnerability Analysis:** A comprehensive inventory of bugs, security vulnerabilities (e.g., reentrancy, access control flaws, integer/math issues, unhandled gas limits), inefficiencies, or logic errors found in the AI's generated output.
 6. **Validation Process & Testing Strategy:** The methods (e.g., unit testing suites like Foundry/Hardhat, static analysis tools like Slither, manual code review, testnet simulation) used to test and validate the AI-assisted codebase.
 7. **Improvements & Refinements:** Functional additions, architectural modifications, and code optimizations made beyond the base AI output.
 8. **Lessons Learned:** Your team's critical evaluation of the AI tools and methodology used—highlighting where AI excelled, where it failed, and recommendations for future AI-driven dApp development.

2.2 Major Issue Breakdown Format

For every major issue or security vulnerability discovered in the AI-generated architecture or codebase, students must provide a detailed breakdown covering:

1. **Why AI made the mistake:** (e.g., hallucinated deprecated Solidity syntax, misunderstood state variable scoping across transactions, lacked context on reentrancy attack vectors, misconfigured front-end web3 provider states).
2. **Why it is problematic:** The potential real-world impact (e.g., risk of drainable funds, stuck/bricked tokens, unauthorized admin privilege escalation, inaccurate price calculation).
3. **How they fixed it:** The specific code refactoring, manual adjustments, or re-prompting/spec updates implemented to resolve the flaw.
4. **Evidence the fix works:** Proof of resolution (e.g., passing unit test execution logs, Slither security check outputs, transaction hashes on testnets, state validation screenshots).

3 Development Resources

There are several online resources (articles, videos, tutorials) to help you with development of dApps on Ethereum. Feel free to consult any online resource. We have put together a set of selected resources in the following GitHub repository:

- **BlockchainCourseNTU GitHub Repository:** <https://github.com/BlockchainCourseNTU/resource/>
 - Check the Quick Access links at the repo to find the main resources quickly.
 - Check the `hello-dapp` page for a detailed introduction to dApp development.
 - Make sure your system is ready with all necessary prerequisites installed.
 - Check the basic and advanced Solidity syntax on the repo before you begin.
- **How does Ethereum work, anyway?** by Preethi Kasireddy: <https://medium.com/@preethikasireddy/how-does-ethereum-work-anyway-22d1df506369>

You can always reach out to the instructors and TAs on the NTULearn discussion board.

4 Evaluation Summary

This assignment is worth **35%** of your total course score. Instructors cannot directly assess individual contributions in team projects. Therefore, peer evaluation is mandatory. Peer evaluations are confidential. Details on score moderation are presented in Section 7.

5 Project Options

5.1 Option 1: Decentralized Exchanges (DEX)

Project in a Nutshell

Build a Minimal Viable Decentralized Exchange (DEX) on Ethereum with a simple front-end web application that supports token listing, trading order submission, order matching/execution, and user self-custody of digital assets.

Background & Problem Statement

Centralized exchanges (e.g., [Coinbase](#), [Binance](#), [Kraken](#), [HTX](#)) hold cryptographic private keys on behalf of users. While convenient, centralized custodial platforms represent major honeypots vulnerable to internal corruption or external hacks (e.g., [Mt.Gox](#)).

Decentralized exchanges solve this fundamental trust assumption. Users retain complete key control via self-custodial [wallets](#) (e.g., MetaMask). The DEX smart contract operates solely as a non-custodial marketplace to match and execute trades directly peer-to-peer.

Feature Requirements

- **Asset Issuer:** Issue new asset tokens adhering strictly to the [ERC20 standard](#).
- **User Features:**
 - Submit buy and sell orders ([limit orders](#) required as baseline).
 - Automatic execution of matched buy/sell orders, supporting partial order fulfillment.
 - (*Bonus 1*) Order cancellation for remaining unfulfilled token amounts.
 - (*Bonus 2*) Batched execution of multi-party order matching.
 - (*Bonus 3*) [Conditional order](#) support.

Tricky Points to Ponder

- How, how often, and who should trigger on-chain execution of matched order pairs?
- Should older unfulfilled orders receive higher priority when matching conditions are identical?

Leading Project References

[Uniswap](#) | [KyberSwap](#) | [0x Project](#) | [CoW Protocol](#) | [Curve Finance](#)

5.2 Option 2: Dutch Auction

Project in a Nutshell

Implement Ethereum smart contracts and a front-end web app for a token launch (STO/ICO) where tokens are bid for using Ether and distributed via a [Dutch auction](#) over a 20-minute window.

Background & Problem Statement

Standard "English auctions" or fixed-price token offerings suffer from severe shortcomings in public Web3 distributions:

1. **Gas Wars & Congestion:** Fixed-price sales trigger massive transaction spikes where users spam high gas fees to secure allocation before sold out.
2. **Distorted Price Discovery:** Pre-set fixed prices often severely overvalue or undervalue tokens.
3. **Bot Manipulation:** Front-running bots routinely drain fixed-price token pools instantly.

A **Dutch Auction** mitigates these issues through uniform market price discovery:

- **Descending Price Mechanism:** Bidding opens at an intentionally high starting price and descends automatically over time based on a predefined decay function.
- **Uniform Clearing Price:** Participants commit ETH capital. When total committed capital meets total token supply, the auction clears.
- **Fair Participant Equity:** *Every winning bidder pays the exact same final clearing price*, regardless of how high their initial bid was. Excess ETH is refunded.

Feature Requirements

- **ERC20 Integration:** Define project tokens using the [ERC20 standard](#).
- **Auction Engine:** Implement Dutch auction mechanics in separate smart contract(s).
- **Time-Bound Elapse:** The auction runs for exactly 20 minutes. Tokens sell out at/above reserve price, or unsold tokens are burned.
- **Token Claims & Refunds:** Distribute minted tokens and automatically process ETH refunds for surplus bid amounts.
- *(Bonus)* Demonstrate reentrancy attack resistance ([Guide](#) / [Practice Repo](#)).

Tricky Points to Ponder

- How to enforce block-based EVM time countdowns and continuous price decay?
 - How to calculate claims when a bid bridges across the clearing threshold?
 - How to process ETH refunds safely against reentrancy vectors?
 - How are unsold tokens permanently burned from supply?
-

5.3 Option 3: Decentralized Domain Registrar

Project in a Nutshell

Build a “.ntu” domain name registrar on an Ethereum testnet using a “commit-and-reveal” bidding process and a web interface for registration and resolution.

Background & Problem Statement

Centralized DNS registrars (e.g., GoDaddy, Namecheap) act as single points of failure with power to censor domains, arbitrarily hijack records, or engage in domain squatting and front-running search queries.

A **Decentralized Domain Registrar** operates transparently on-chain. To prevent front-running on public blockchain mempools—where front-running MEV bots copy bid transactions with higher gas fees to steal domain names—registrars utilize a **Commit-and-Reveal (Blind Auction)** mechanism:

1. **Commit Phase:** Bidders submit a hidden cryptographic commitment Hash(domain, bid, salt) along with ETH deposit. External observers cannot determine which domain is being targeted or the bid value.
2. **Reveal Phase:** Bidders reveal domain, bid, and salt. The contract verifies the hash against prior commitment and assigns ownership to the highest valid bidder.

Feature Requirements

- **Domain Bidding:** Allow any address to bid for unregistered “.ntu” domains.
- **Commit-and-Reveal Protocol:** Implement blind auction logic adhering to [Solidity standards](#) (accelerated to 2–3 minute phases for testing).
- **Resolution:** Support forward resolution (Domain → Ethereum Address).
- **Frontend UI:** Search domain status, execute commit/reveal steps, and send ETH directly to a registered “.ntu” name.
- (*Bonus*) Reverse resolution (Ethereum Address → Primary “.ntu” domain).

Tricky Points to Ponder

- How to handle unrevealed commitments and penalize non-revealing bidders?
 - How to prevent input data inspection from exposing domains during commit phases?
-

5.4 Option 4: Prediction Market

Project in a Nutshell

Build a decentralized prediction market where users create, trade, and resolve speculative markets on future real-world events.

Background & Problem Statement

Centralized forecasting institutions often fail due to lack of direct financial accountability. Decentralized prediction markets leverage the *Wisdom of the Crowd*:

- **Incentive Alignment:** Participants trade conditional outcome shares (e.g., YES/NO). Real-time share prices reflect collective probability estimates.
- **Decentralized Resolution:** Eliminates centralized sportsbook rake fees and unilateral account restrictions.
- **The Oracle Problem:** Smart contracts require secure mechanisms (e.g., trusted arbitrators, reporter staking) to report real-world outcomes accurately.

Feature Requirements

- **Market Creation:** Open new prediction topics with outcome options, timestamps, and arbitrator identities.
- **Trading & Minting:** Place bets, receive outcome tokens, and dynamically derive market odds.
- **Resolution & Payouts:** Arbitrator submits final outcome; winners redeem shares for rewards.
- *(Bonus 1)* Secondary derivative market trading prior to event resolution.
- *(Bonus 2)* Multi-asset collateral support (e.g., [Tether USD](#)).
- *(Bonus 3)* Reputation-weighted voting mechanisms.

Leading Project References

Polymarket | [Omen](#) | [Augur Docs](#)

5.5 Option 5: Bring Your Own Project (BYOP)

Students may propose custom dApp ideas subject to coordinator approval.

- Submit a 1-page proposal detailing problem statement, MVP features, and technical scope.
- Inspiration sources: [DeFi SoK Paper](#), Vitalik Buterin's "[Things Outside DeFi](#)", [Quadratic Voting](#), or [Gitcoin Bounties](#).

6 Assessment Criteria

Criteria	Fail (0–40%)	Pass (41–74%)	High Standard (75–100%)
Problem Identification & Solution Design	Incorrect problem definition; solution unrelated to core requirements.	Correct problem identification; practical and logical blockchain solution design.	Precise problem identification; technically accurate, complete, and practical Web3 solution.
Development & AI Audit Quality	Ad hoc coding; no critical analysis of AI output or AI tooling workflow.	Logical code implementation; satisfactory AI Audit Report analyzing prompt design and bug fixes.	High-competence code; rigorous evaluation of AI agents/tools, prompt specs, and vulnerability remediation.
Codebase, Docs & UX	Disorganized codebase, missing documentation and AI Audit Report.	Structured code, adequate comments, functional UI, and complete AI Audit Report.	Production-quality codebase, comprehensive documentation, sleek UX, and exemplary AI Audit.

Table 1: Assessment Rubric for Development Project (35% total mark)

7 Peer Evaluation within Team

Peer evaluation is mandatory to moderate individual grades within group submissions.

Rating	Standard
9 – 10	Outstanding contributions and effort; took leadership role.
7 – 8	Equal and appropriate contributions; active teamwork.
4 – 6	Sufficient contribution; could have exerted greater effort.
1 – 3	Minimal contribution toward project completion.
0	Zero effort or contribution.

Table 2: Peer Evaluation Rating Scale

Grade Calculation Formula

Let R be the student's Mean Peer Rating:

- If $R \geq 8.0$: Student receives 100% of the team project score.
- If $2.0 \leq R < 8.0$: Student receives $(20\% + R \times 10\%)$ of the team project score.
- If $R < 2.0$: Case subject to formal investigation (may result in 0%).

The course coordinator reserves the right to review/revise each peer evaluation rating for your team in case of questionable circumstances, which include, but are not limited to, acts of bias, discrimination or malice.